

Методичка 18. Основы SQL и базы данных

1. Зачем нужна эта тема

Базы данных нужны, чтобы хранить данные удобно, надёжно и долго.

До этого данные можно было хранить:

- в переменных;
- в списках;
- в файлах;
- в объектах.

Но у этих способов есть ограничения.

Например, если нужно хранить много товаров, работников, машин, пользователей или заказов, обычный текстовый файл становится неудобным. Сложно быстро искать данные, сортировать, фильтровать, обновлять отдельные записи.

База данных помогает:

- хранить данные в таблицах;
- быстро искать нужные строки;
- сортировать данные;
- фильтровать по условиям;
- добавлять новые записи;
- изменять существующие записи;
- удалять ненужные записи;
- считать суммы, средние значения и количество записей.

SQL — это язык запросов к базе данных.

Он нужен, чтобы программа могла сказать базе данных:

- создай таблицу;
- добавь запись;
- покажи все записи;

- найди записи по условию;
- измени запись;
- удали запись.

2. Главная идея темы

База данных похожа на набор таблиц.

Таблица похожа на таблицу в Excel:

- есть столбцы;
- есть строки;
- каждая строка — отдельная запись;
- каждый столбец — отдельное поле.

Например, таблица `cars` может хранить машины:

id	brand	model	price	1	2
1	BMW	X5	8000000		
2	Toyota	Camry	3500000		

База данных — место для структурированного хранения данных.

Таблица — набор строк и столбцов.

Строка / запись — один объект в таблице.

Столбец / поле — характеристика объекта.

SQL — язык запросов к базе данных.

Запрос — команда к базе данных.

Первичный ключ — уникальный идентификатор записи, часто поле `id`.

3. Базовый синтаксис

Создание таблицы

```
CREATE TABLE cars (  
    id INTEGER PRIMARY KEY,  
    brand TEXT,  
    model TEXT,  
    price INTEGER  
);
```

Что происходит:

- создаётся таблица `cars` ;
- у неё есть поля `id` , `brand` , `model` , `price` ;
- `id` — уникальный номер записи;
- `TEXT` — текст;
- `INTEGER` — целое число.

Добавление данных

```
INSERT INTO cars (brand, model, price)
VALUES ('Toyota', 'Camry', 3500000);
```

Запрос добавляет новую машину в таблицу.

Получение всех данных

```
SELECT * FROM cars;
```

`SELECT` получает данные. `*` означает “все столбцы”.

Получение отдельных столбцов

```
SELECT brand, model FROM cars;
```

Запрос показывает только бренд и модель.

Фильтрация

```
SELECT * FROM cars
WHERE price > 3000000;
```

`WHERE` задаёт условие.

Сортировка

```
SELECT * FROM cars
ORDER BY price DESC;
```

`ORDER BY` сортирует данные. `DESC` — по убыванию. `ASC` — по возрастанию.

Обновление данных

```
UPDATE cars
SET price = 3300000
WHERE id = 1;
```

Запрос изменяет цену у записи с `id = 1`.

Удаление данных

```
DELETE FROM cars
WHERE id = 1;
```

Запрос удаляет запись с `id = 1`.

4. Разбор темы от простого к сложному

Шаг 1. Таблица как структура

Сначала нужно понять, какие данные хранятся.

Например, для машин:

- бренд;
- модель;
- год;
- цена.

```
CREATE TABLE cars (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    year INTEGER,
    price INTEGER
);
```

Хорошая таблица хранит однотипные данные.

Шаг 2. Добавление записей

```
INSERT INTO cars (brand, model, year, price)
VALUES ('Toyota', 'Camry', 2020, 3500000);

INSERT INTO cars (brand, model, year, price)
VALUES ('BMW', 'X5', 2021, 8000000);
```

Каждый `INSERT` добавляет одну строку.

Шаг 3. Просмотр всех записей

```
SELECT * FROM cars;
```

Этот запрос полезен для проверки: какие данные уже есть в таблице.

Шаг 4. Выбор нужных полей

```
SELECT brand, model FROM cars;
```

Если не нужны все данные, можно выбрать только часть столбцов.

Шаг 5. Фильтрация через WHERE

```
SELECT * FROM cars  
WHERE year >= 2020;
```

Запрос показывает только машины от 2020 года.

Шаг 6. Несколько условий

```
SELECT * FROM cars  
WHERE year >= 2020 AND price <= 4000000;
```

`AND` означает, что должны выполняться оба условия.

```
SELECT * FROM cars  
WHERE brand = 'BMW' OR brand = 'Toyota';
```

`OR` означает, что достаточно одного условия.

Шаг 7. Сортировка

```
SELECT * FROM cars  
ORDER BY price ASC;
```

Сортировка по цене от меньшей к большей.

```
SELECT * FROM cars  
ORDER BY price DESC;
```

Сортировка по цене от большей к меньшей.

Шаг 8. Ограничение количества записей

```
SELECT * FROM cars  
LIMIT 5;
```

Запрос показывает только первые 5 записей.

```
SELECT * FROM cars  
LIMIT 5 OFFSET 5;
```

Запрос пропускает первые 5 записей и показывает следующие 5.

Шаг 9. Агрегатные функции

```
SELECT COUNT(*) FROM cars;
```

Количество записей.

```
SELECT AVG(price) FROM cars;
```

Средняя цена.

```
SELECT MAX(price) FROM cars;
```

Максимальная цена.

```
SELECT SUM(price) FROM cars;
```

Сумма цен.

Шаг 10. Обновление записи

```
UPDATE cars  
SET price = 3000000  
WHERE id = 2;
```

Важно всегда писать `WHERE` , иначе можно изменить все строки таблицы.

Шаг 11. Удаление записи

```
DELETE FROM cars
WHERE id = 2;
```

Важно всегда писать `WHERE` , иначе можно удалить все строки таблицы.

Шаг 12. Связь с Python

В Python можно подключаться к базе данных и выполнять SQL-запросы.

Упрощённый пример:

```
import sqlite3

connection = sqlite3.connect("shop.db")
cursor = connection.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY,
    title TEXT,
    price INTEGER
)
""")

connection.commit()
connection.close()
```

Здесь Python создаёт файл базы данных `shop.db` и таблицу `products` .

5. Частые ошибки учеников

Ошибка 1. Забывают точку с запятой

Неправильно:

```
SELECT * FROM cars
```

Почему это может быть проблемой:

Во многих SQL-редакторах точка с запятой показывает конец запроса.

Правильно:

```
SELECT * FROM cars;
```

Ошибка 2. Пишут строковые значения без кавычек

Неправильно:

```
INSERT INTO cars (brand, model, price)
VALUES (Toyota, Camry, 3500000);
```

Почему это ошибка:

Текстовые значения должны быть в кавычках.

Правильно:

```
INSERT INTO cars (brand, model, price)
VALUES ('Toyota', 'Camry', 3500000);
```

Ошибка 3. Путают **SELECT** и **INSERT**

Неправильно:

```
SELECT INTO cars (brand, model)
VALUES ('BMW', 'X5');
```

Почему это ошибка:

SELECT получает данные. **INSERT** добавляет данные.

Правильно:

```
INSERT INTO cars (brand, model)
VALUES ('BMW', 'X5');
```

Ошибка 4. Забывают **WHERE** в **UPDATE**

Неправильно:

```
UPDATE cars
SET price = 1000000;
```

Почему это опасно:

Так изменится цена у всех машин.

Правильно:


```
UPDATE cars
SET price = 1000000
WHERE id = 1;
```

Ошибка 5. Забывают WHERE в DELETE

Неправильно:

```
DELETE FROM cars;
```

Почему это опасно:

Так удалятся все записи из таблицы.

Правильно:

```
DELETE FROM cars
WHERE id = 1;
```

Ошибка 6. Путают **AND** и **OR**

Неправильно:

```
SELECT * FROM cars
WHERE price < 1000000 AND price > 5000000;
```

Почему это ошибка:

Цена не может одновременно быть меньше 1 000 000 и больше 5 000 000.

Правильно:

```
SELECT * FROM cars
WHERE price < 1000000 OR price > 5000000;
```

Ошибка 7. Неправильно сортируют

Неправильно:

```
SELECT * FROM cars
WHERE ORDER BY price;
```

Почему это ошибка:

`WHERE` и `ORDER BY` — разные части запроса. После `WHERE` должно быть условие.

Правильно:

```
SELECT * FROM cars
ORDER BY price;
```

Или с условием:

```
SELECT * FROM cars
WHERE price > 1000000
ORDER BY price;
```

6. Мини-примеры для закрепления

Пример 1. Создать таблицу

```
CREATE TABLE workers (
    id INTEGER PRIMARY KEY,
    name TEXT,
    salary INTEGER
);
```

Закрепляется структура таблицы.

Пример 2. Добавить запись

```
INSERT INTO workers (name, salary)
VALUES ('Анна', 90000);
```

Закрепляется `INSERT`.

Пример 3. Вывести все записи

```
SELECT * FROM workers;
```

Закрепляется базовый `SELECT`.

Пример 4. Фильтрация

```
SELECT * FROM workers  
WHERE salary >= 100000;
```

Закрепляется `WHERE` .

Пример 5. Сортировка

```
SELECT * FROM workers  
ORDER BY salary DESC;
```

Закрепляется `ORDER BY` .

Пример 6. Количество записей

```
SELECT COUNT(*) FROM workers;
```

Закрепляется агрегатная функция `COUNT` .

Пример 7. Средняя зарплата

```
SELECT AVG(salary) FROM workers;
```

Закрепляется `AVG` .

Пример 8. Обновление

```
UPDATE workers  
SET salary = 120000  
WHERE id = 1;
```

Закрепляется `UPDATE` и важность `WHERE` .

Пример 9. Удаление

```
DELETE FROM workers  
WHERE id = 1;
```

Закрепляется `DELETE` и важность `WHERE` .

7. Практические задания

Лёгкий уровень

1. Создать таблицу `students` с полями `id` , `name` , `age` .
2. Добавить в таблицу `students` три записи.
3. Вывести все записи из таблицы `students` .
4. Вывести только имена учеников.
5. Вывести учеников старше 15 лет.

Средний уровень

1. Создать таблицу `products` с полями `id` , `title` , `price` , `count` .
2. Добавить 5 товаров.
3. Вывести товары дороже 1000.
4. Отсортировать товары по цене по возрастанию.
5. Посчитать количество товаров.
6. Найти среднюю цену товаров.
7. Обновить цену товара с конкретным `id` .

Повышенный уровень

1. Создать таблицу `workers` с полями `id` , `name` , `position` , `salary` .
2. Вывести работников с зарплатой от 80 000 до 150 000.
3. Вывести топ-3 работников по зарплате.
4. Посчитать сумму всех зарплат.
5. Удалить работника по `id` .
6. Написать Python-код, который подключается к SQLite и создаёт таблицу `products` .

8. Контрольные вопросы

1. Что такое база данных?
2. Что такое таблица?
3. Что такое строка в таблице?
4. Что такое столбец в таблице?
5. Для чего нужен `id` ?
6. Что делает `CREATE TABLE` ?
7. Что делает `INSERT INTO` ?
8. Что делает `SELECT` ?
9. Для чего нужен `WHERE` ?
10. Чем `AND` отличается от `OR` ?
11. Что делает `ORDER BY` ?

12. Что делает `LIMIT` ?
13. Что делает `COUNT(*)` ?
14. Почему опасно писать `UPDATE` без `WHERE` ?
15. Почему опасно писать `DELETE` без `WHERE` ?

9. Краткий конспект в конце

Главная идея темы: база данных хранит данные в таблицах, а SQL позволяет этими данными управлять.

Основные команды:

```
CREATE TABLE cars (  
    id INTEGER PRIMARY KEY,  
    brand TEXT,  
    price INTEGER  
);
```

```
INSERT INTO cars (brand, price)  
VALUES ('Toyota', 3500000);
```

```
SELECT * FROM cars;
```

```
SELECT * FROM cars  
WHERE price > 3000000;
```

```
SELECT * FROM cars  
ORDER BY price DESC;
```

```
UPDATE cars  
SET price = 3000000  
WHERE id = 1;
```

```
DELETE FROM cars  
WHERE id = 1;
```

Важно запомнить:

- база данных состоит из таблиц;
- таблица состоит из строк и столбцов;
- `SELECT` получает данные;
- `INSERT` добавляет данные;

- `UPDATE` изменяет данные;
- `DELETE` удаляет данные;
- `WHERE` задаёт условие;
- `ORDER BY` сортирует;
- `COUNT` , `SUM` , `AVG` , `MAX` считают данные;
- `UPDATE` и `DELETE` почти всегда должны быть с `WHERE` .

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- понимать, что такое база данных;
- понимать, что такое таблица;
- создавать простую таблицу;
- добавлять записи;
- получать записи через `SELECT` ;
- выбирать отдельные столбцы;
- фильтровать данные через `WHERE` ;
- использовать `AND` и `OR` ;
- сортировать данные через `ORDER BY` ;
- ограничивать вывод через `LIMIT` ;
- использовать `COUNT` , `SUM` , `AVG` , `MAX` ;
- обновлять записи через `UPDATE` ;
- удалять записи через `DELETE` ;
- понимать опасность `UPDATE` и `DELETE` без `WHERE` ;
- понимать базовую связь SQL и Python.